

Bridging Go to z/OS via Direct Assembly

Mentorship roadmap, endgame target: WTO from Go assembly

Author	Mert Efe Şensoy · TED University · IBM Z Student Ambassador · IBM Champion 2026
Mentor	Jürgen Holtz · IBM
Working repo	zbridge-asm-labs

Thesis

Go on z/OS is production-ready as a runtime but library-poor at the boundary with native z/OS services. IBM's `ibmruntimes/go-recordio` module proves the architectural pattern is viable: Go assembly can navigate z/OS control blocks, switch between AMODE 31 and AMODE 64, allocate below-the-bar storage, convert UTF-8 to EBCDIC, and call MVS service entry points directly without `cgo`. That repository is BSD-3-Clause licensed and published, but it covers only the IEFSSREQ subsystem request macro. Operator-facing services like WTO have no public Go-assembly implementation today.

This roadmap targets that gap. The endgame deliverable is a single Go function, exported as `WTO(message string) error`, implemented in pure Go assembly for s390x, that successfully writes a message to the z/OS operator console via SVC 35. No `cgo`. No Language Environment dependency. The implementation reuses the architectural primitives `go-recordio` established, validates them on a different service class, and produces a referenceable example for what `go-recordio`'s IEFSSREQ walkthrough does not cover: an unauthorized, problem-state, operator-facing call.

Why WTO is the right endgame target

Four reasons make WTO the right choice for a thesis demonstration of the broader bridging vision.

- **Problem-state accessible.** SVC 35 is callable from problem state, key 8, without APF authorization. Unauthorized WTO messages carry an at-sign prefix on the console, which is a cosmetic limitation, not a functional block. The architectural payoff is full.
- **Visible side effect.** Success is unambiguous. The message either appears on the operator console or it does not. No abstract correctness criterion to defend.
- **Architectural completeness.** WTO exercises every primitive the broader bridging vision needs: parameter list construction, below-the-bar storage allocation, EBCDIC conversion, SVC dispatch, and return code evaluation in R15. Once WTO works, extending to

Name/Token, ENQ/DEQ, and other problem-state SVC-based services is mechanical translation of the same pattern.

- **Documented gap.** Research confirms no public Go-assembly WTO implementation exists. Python has it via Ambitus's pyzkiln. Java has it via JZOS. Go does not.

The architectural pattern, in detail

From go-recordio's `utils/utils.s`, established May 2025, the pattern decomposes into five primitives. WTO uses all of them except the CVT navigation step, since the SVC dispatcher handles the kernel-side lookup. This makes WTO a strict subset of go-recordio's pattern, which is what makes it the right next step rather than a parallel exploration.

Primitive	What it does	Where it lives
CVT → JESCT navigation	Locate service entry points via the standard z/OS control block chain	Pure Go assembly
SAM31 / SAM64	Switch addressing mode around the BALR call, required because most MVS macros are 31-bit	Go assembly directives wrapping BALR
Malloc31	Allocate storage below the 2 GB bar, required for legacy MVS parameter lists	One of the few necessary LE touchpoints, exposed via go-recordio's helper
AtoE / EtoA	UTF-8 ↔ EBCDIC IBM-1047 conversion via 256-byte lookup table. The s390x TR instruction collapses the entire amd64 lookup loop into one hardware op	Pure Go assembly
SVC dispatch	The SVC n instruction itself, with parameters pre-loaded in R0 and R1, return code read from R15	Pure Go assembly

Phased plan

The plan is six phases. Phase 0 was deferred when two mentor meetings slipped (your quarterly report on your side, midterms on mine), so its content has been folded into Phases 1 and 2 rather than treated as a standalone block. The detailed write-ups for each phase follow.

Phase	Focus	Window	Hardware
0	Foundation: assembler theory + Go asm reading	Folded into 1+2	Laptop only
1	Go assembly on x86: toolchain checkpoint + 5 exercises	Now through late June	amd64 laptop
1b	Port the same 5 exercises to s390x	Late June – mid July	LinuxONE Community Cloud
2	go-recordio utils.s deep annotation	Mid July – early August	Reading only
3	WTO implementation on z/OS via SVC 35	Mid August – late September	z/OS USS
4	Stretch: WTOR, Name/Token, polish for community	Q4 2026 and beyond	z/OS USS

Phase 0 · Foundation

Status: deferred, absorbed into Phases 1 and 2.

The original target was 7 April to 27 April. Compressed by mentor meeting slippage, midterms, and part-time work. Rather than holding the rest of the plan hostage to a completed Phase 0, the foundational components are being interleaved into the hands-on phases:

- Mainframe Skills Depot Assembler Part 1 and Part 2 (free courses): in progress, target completion alongside Phase 1.
- PC/370 textbook exercises from the toolchain you provided (Bill Qualls + Don Higgins emulator): selected exercises HELLO.MLC through TEACH4 series serve as foundational practice during Phase 1.
- Go Assembler Guide at go.dev/doc/asm: completed before Phase 1's first exercise.
- Reading 3+ Go s390x assembly files from the standard library: scheduled inside Phase 2.

Revised deliverable: the s390x Assembly Cheat Sheet (2-3 pages) ships at the end of Phase 2 rather than the end of Phase 0.

Phase 1 · Go assembly on x86

Status: in progress, scoped tight for the next meeting.

The first deliverable is the add exercise inside a new repository, `zbridge-asm-lab/add/`. One function, one assembly file, one test, one benchmark. This is a toolchain checkpoint, not a feature. Built deliberately small because TEDU finals run through early June and the meeting needs a concrete artifact regardless of the broader plan's progress.

The full Phase 1 deliverable (after finals) is five exercises in the same repo, each in its own subdirectory, with x86-to-s390x mapping notes baked into the README of each.

Exercise	What it proves	Maps to what on s390x
add	Plan 9 syntax, TEXT directive, register passing, frame size discipline, toolchain end-to-end	Same instruction class, different register names
regs	Reading hardware state (BP, SP), stack frame layout	Different register file (R0-R15, F0-F15), distinct conventions for R13, R14, R15
bytecmp	Memory instructions, loop control, condition codes, benchmark discipline	CLC family for byte comparison
ebcdic	Table-driven byte translation via 256-byte lookup, the critical primitive for every MVS parameter list	TR instruction collapses the entire lookup loop into one hardware op
syscall-linux	Issuing a SYSCALL instruction from Go assembly, with parameters in registers and return value evaluation	Same conceptual primitive as SVC, different opcode and trap mechanism

Deliverable: Public GitHub repo `zbridge-asm-lab` with each exercise self-contained, a root README that catalogs the exercises, and per-exercise documentation that calls out the s390x equivalents.

Realistic completion: late June, immediately after finals.

Phase 1b · Port to s390x via LinuxONE

Status: blocked on access.

The port is mechanical translation once Phase 1 is solid. The blocker is LinuxONE Community Cloud access. I will provision an instance the day Phase 1 ships and aim to have all five exercises running on s390x within two weeks.

Deliverable: A second commit series in the same repo, with `_s390x.s` files alongside the `_amd64.s` files. The `ebcdic` exercise gets a side-by-side comparison: the `amd64` lookup loop versus the `s390x` TR instruction, with benchmark numbers attached.

Open question for you: *is LinuxONE Community Cloud sign-up the right path, or is there a faster route via the IBM ambassador program?*

Phase 2 · go-recordio deep annotation

Status: not started, originally May 19 to June 1, now slipping to mid-July.

This is the most important pre-z/OS phase. The work is reading `ibmruntimes/go-recordio/utls/utls.s` line by line and annotating every instruction with what it does, why it does it, and what the corresponding HLASM macro would have generated. Specific deliverables:

- Annotated walkthrough of `utls.s`, instruction by instruction, in a format suitable for publication.
- A diagram of the `CVT` → `JESCT` → `SSREQ` navigation path with byte offsets called out.
- Documentation of the `SAM31/SAM64` switching pattern around the `BALR` call.
- An identification of which patterns in `utls.s` are `SSREQ`-specific and which transfer directly to `WTO`.

Open question for you: *would you or Lucas be willing to review the annotated walkthrough? Nobody has published this document, and a reviewed version would be a contribution to the ecosystem on its own, independent of the WTO endgame.*

Phase 3 · WTO implementation on z/OS

Status: dependent on Phase 2 completion and z/OS access.

The endgame phase. Three subphases.

3a · Toolchain validation

Cross-compile the Phase 1b exercises with GOOS=zos GOARCH=s390x and run them on z/OS USS. This validates that what works on s390x Linux also works on z/OS without modification, which is the assumption Phase 1b is built on.

3b · WTO scaffold

Implement WTO in Go assembly using the architectural primitives from go-recordio. The function signature is WTO(message string) error. The implementation steps:

1. Allocate a parameter buffer below the bar via Malloc31.
2. Translate the message from UTF-8 to EBCDIC IBM-1047 via AtoE.
3. Construct the WTO parameter list (2-byte length header followed by EBCDIC message text, per IBM docs).
4. Load R1 with the parameter list address.
5. Issue SVC 35.
6. Read R15 for the return code and translate to a Go error.

3c · Validation and documentation

Confirm the message appears on the operator console. Document every step. Compare latency against the same operation performed via __console2() through cgo, with benchmark numbers attached.

Deliverable: A public Go module, BSD-3-Clause to match go-recordio's licensing, with implementation, tests, and a findings document explaining what worked, what did not, and what required workarounds.

Realistic completion: late summer 2026, dependent entirely on z/OS access timing.

Open question for you: *what is the realistic timeline for z/OS access? Wazi-aaS, a shared LPAR, the more open Z environment Lucas mentioned, or something else?*

Phase 4 · Stretch deliverables (only after WTO ships)

Three candidates, in order of recommended priority:

- **WTOR.** Write To Operator with Reply. Adds asynchronous two-way console communication, using the same SVC 35 mechanism plus an Event Control Block for the reply. Architecturally novel because the Go wrapper has to bridge the legacy mainframe asynchronous event model to Go's channels and goroutines, instead of blocking the OS thread with a WAIT macro.
- **Name/Token services.** IEAN4RT (the 64-bit retrieval variant) and IEANTCR. Standard MVS CALL linkage rather than direct SVC, but the architectural pattern is parallel: parameter list construction, EBCDIC conversion, calling convention discipline.
- **Go-native EBCDIC utility package.** Extract the conversion routines from the exercises above, polish, test, publish as a standalone Go module for the broader Go-on-z/OS community. Lower technical novelty than the SVC work, but the highest immediate utility for other developers.

A Go module named something like go-mvscalls or similar, covering two or three services in this family, would be the natural framing if the work scales beyond WTO.

Risks and dependencies

- **z/OS access timing.** The single biggest risk. Hercules is not legal for z/OS. Z Xplore is not flexible enough for custom assembly work. Wazi-aaS requires IBM Cloud entitlement. The mitigation is to push every phase that does not require z/OS as far as possible before the access constraint bites, which is exactly the structure of Phases 0 through 2 and subphase 3a.
- **Mentor availability.** Two cancellations in the spring (your quarterly report, my midterms) compressed the schedule. Going forward, I would propose a fixed monthly cadence with explicit pre-meeting deliverables on my side, so that if a meeting slips, the next one has a clear catch-up agenda rather than starting over.
- **Finals.** TEDU finals run through early June. Phase 1 is scoped down to the add toolchain checkpoint specifically because of this. Real Phase 1 work resumes mid-June.
- **Sustained time budget.** Continuing part-time at Turkish Airlines Revenue Management, 15-20 hours per week. The 5-8 hours per week budget for this project is realistic and sustained, not optimistic.

What need to be worked on

1. Sanity check on the WTO endgame target. Is this the right scope for a student-level project on this timeline, or should the target be downscaled (WTO via __console2() through cgo first, then the SVC-direct version as a stretch)?
2. The z/OS access path. Which option is realistic on what timeline?
3. Willingness to review the Phase 2 annotated walkthrough of go-recordio when it ships.
4. Contacts at IBM who have done Go assembly work on s390x. The 95-member IBM Community group for Go on z/OS exists, but specific names doing low-level work would help.
5. Directions to avoid. Things that are politically sensitive within IBM, technical dead ends, or scope traps you have seen past students fall into.

This roadmap is a living document. The phase definitions are stable, the timeline is provisional, and the open questions are listed honestly. I would rather over-communicate the dependencies and unknowns than ship a plan that pretends everything is solved.